CENG-567
REINFORCEMENT LEARNING

Lecture-3

Contextual Bandits

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

What is MAB?

A simplified RL setting: single state, repeated decisions

Goal: maximize cumulative reward

Core challenge: **Exploration vs. Exploitation**

Exploration Strategies Learned

A/B/n Testing: fixed, non-adaptive

ε-Greedy: adaptive but still random exploration

UCB (Upper Confidence Bounds):

Idea: Value = Estimate + Uncertainty

Selects actions optimistically → efficient directed exploration

Thompson Sampling:

Bayesian approach

Samples from posterior → natural exploration using uncertainty

Key Takeaway:
Different strategies balance exploration/exploitation differently; UCB and Thompson Sampling outperform naive methods.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Comparing the Methods

Method	Exploration Style	Notes
A/B/n	Fixed	Very inefficient
ε-Greedy	Random	Needs ε tuning
UCB	Pull frequency	Tunable, principled exploration
Thompson	Bayesian sampling	Strong empirical performance

Using Context in MABs

- A more advanced version of the multi-armed bandit is the **contextual bandit** (**CB**) problem, where decisions are tailored to the context they are made in.
- In previous lectures, we identified the best-performing ad in an online advertising scenario. In doing so, we did not use any information about, **for instance, the user persona, age, gender, location, or previous visits**, which would have increased the likelihood of a click.
- CBs allow us to leverage this information, which means, they play a role in central role in commercial personalization and recommendation applications.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Contextual Bandits (CBs) are NOT Multistep RL

- **State in RL:**
  - Actions influence *future states*.
  - Example: **Tic-tac-toe** — each move changes the board and affects all future possibilities.
- **Context in CB:**
  - The agent **observes context → chooses action → gets reward**.
  - The **next context does *not*** depend on the current action.
  - No state transitions, no long-term planning.
- **Key Point:**
  CB is simpler than multi-step RL, but still powerful and widely used (e.g., ads, recommendations, personalization).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why Do We Need Function Approximation in CB?

Goal in CB/MAB:
Learn **action values** $Q(a \mid context)$ from observations.

Example Context (User Info):

Device: *Mobile / Desktop*

Location: *Domestic / International*

This leads to **4 separate contexts**, meaning **4 separate MAB problems.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why Do We Need Function Approximation in CB?

When we add more context variables (e.g., age, time, device, location, etc.):

🚧 **Challenges emerge:**

Data fragmentation

Too many context combinations

Not enough samples for each (e.g., *Mobile, International, age 57*)

Exponential growth of context space

+1 feature → ×N more contexts

Quickly becomes intractable to store and learn a table of Q-values

No knowledge sharing between similar contexts

Age 56 and age 57 treated as unrelated, even though they are similar

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Using Function Approximation for Context

- Function approximations allow us to model the dynamics of a process from which we have observed data, such as contexts and ad clicks.
- Consider an online advertising scenario with **five different ads** (A, B, C, D, and E), with the **context** comprising **user device**, **location**, and **age**.
- Our agent will learn **five different Q (quality) functions**, one **per ad**, each **receiving a context** of x = [device, location, age] , and **return the action value estimate**.

Figure 3.1 – We learn a function for each action that receives the context and returns the action value

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Using Function Approximation for Context

With function approximators, the contextual bandit becomes a **supervised learning problem**:

For **each action** we learn a model mapping

$$x \rightarrow Q(a \mid x)$$

Possible models:

- ✅ Logistic Regression
- ✅ Neural Networks (single network can output **all** $Q(a)$values at once)

After learning $Q(a \mid x)$, we can **still use** the same **exploration strategies**:

- ε-greedy
- UCB
- Thompson Sampling

Key Benefit:
Works with **high-dimensional, continuous, and realistic context data**, unlike the table-based approach.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap: Logistic Regression

Goal: Predict **binary outcomes** (e.g., *click / no-click*, *spam / not spam*, *disease / no disease*).

Unlike linear regression, logistic regression predicts a **probability**, not a raw continuous value

Uses the **sigmoid (logistic) function** to squeeze predictions into the interval **(0, 1)**

$$\hat{y} = P(Y = 1 \mid x) = \sigma(w^T x + b)$$

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Decision rule:

$$\hat{y} = \begin{cases} 1, & \textit{if } \hat{y} \geq 0.5 \\ 0, & \textit{otherwise} \end{cases}$$

✅ Outputs a probability
✅ Works well for linearly separable classes

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap: Logistic Regression

Training objective: Find parameters w, bthat make predictions match labels.

Uses **Log-Loss (Cross-Entropy Loss):**

Penalizes confident wrong predictions heavily

Convex loss → **guaranteed global minimum** (Convexity implies **no local minima**, only a **single global minimum**)

Learning method: Gradient Descent

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case study – contextual online advertising with synthetic user data

Assume that the true user click behavior follows a logistic function:

$$p_a(x) = \frac{1}{1 + e^{-f_a(x)}}$$

$$f_a(x = [device, location, age]) = \beta_0^a + \beta_1^a \cdot device + \beta_2^a \cdot location + \beta_3^a \cdot age$$

Here, $\boldsymbol{p_{a(x)}}$ **is the probability of a user click** when the context is x and ad a is shown.

Also, let's assume that *device* is 1 for mobile and 0 otherwise; and *location* is 1 for US and 0 otherwise.

- ❑ This behavior, particularly the $\boldsymbol{\beta}$ **parameters, is unknown to the advertiser**, which they **will try to uncover**.
- ❑ Note the a superscript in β_i^a, which denotes that the impact of these factors on **user behavior is potentially different for each ad**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case study – contextual online advertising with synthetic user data

See '**Contextual bandits.ipynb**' file in the Teams course page.

- We first create `UserGenerator` class to simulate user dynamics. And set some true β parameters, which the advertiser (the agent) will try to learn.
- Then define the methods to generate a click or no clicks given the user context.
- After that define a method that will generate users with different contexts randomly (US user with 60% chance, mobile device with 80% chance, user ages vary between 10 and 70, with a mean age of 34).
- Finally, create an object instance to generate users and visualize user behavior.

Case study – contextual online advertising with synthetic user data

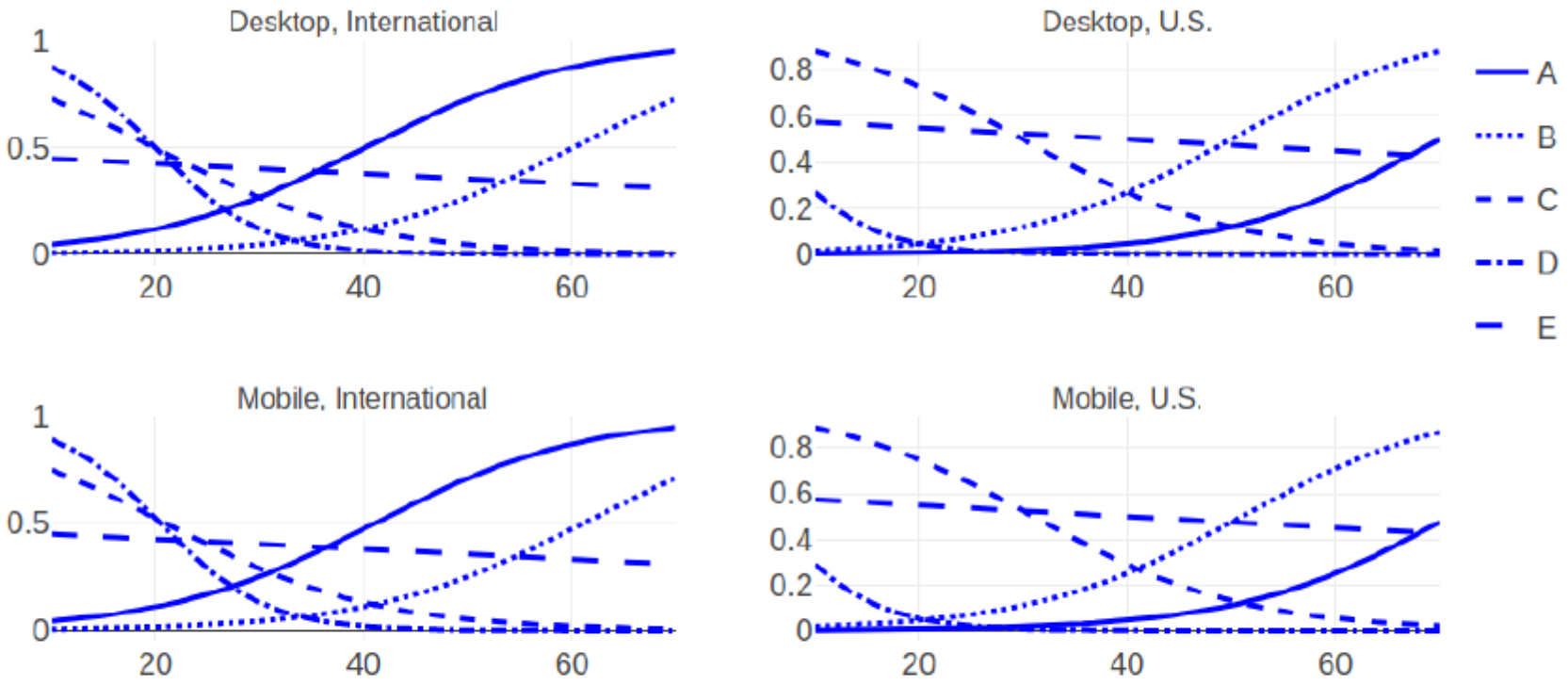


Figure 3.2 – Comparison of the true ad click probabilities given the context (x axis: age, y axis: click probability)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Function approximation with regularized logistic regression

Goal:
Update a model that estimates $Q(a \mid context)$and selects the best ad using an exploration strategy.

Function Approximators commonly used

Logistic Regression (our focus)

Decision Trees / Random Forest

Neural Networks

Exploration Strategies we keep using

ε-Greedy

Upper Confidence Bounds (UCB)

Thompson Sampling

Practical:
In real systems, **models are updated in batches**, not after every user click (high traffic scenario).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Function approximation with regularized logistic regression

We assume **CTR ≈ logistic model**, so we choose a logistic regression–based function approximator that:

- Supports **batch updates**
- Works with **context features**
- Allows **Bayesian/Thompson sampling** by maintaining parameter uncertainty

This leads us to **Regularized Logistic Regression with Posterior Updates** (Chapelle et al., 2011), which lets us:

Maintain a **distribution over weights**, not a single estimate

Use it for **probabilistic action selection**

Goal: Enable Thompson Sampling + scalable learning.

Key Ideas in the Regularized LR Algorithm

- **Approximates** the posterior distribution on the **model weights** by a **Gaussian distribution**. This allows us to use the posterior distribution as the prior in the next batch, and also use **Gaussian for the likelihood function**, since the Gaussian family is conjugate to itself.
- Uses a diagonal covariance matrix for the weights, meaning we **assume** the **weights are not correlated**.
- Uses **Laplace approximation to obtain the mean and the variance estimates** of the **weight distributions**, one of the common methods in statistics to estimate the posterior parameters from observed data if the posterior is assumed to be Gaussian.

You can learn more about Laplace approximation for computing the posterior mean at https://bookdown.org/rdpeng/advstatcomp/laplace-approximation.html.

Implementing the Regularized LR

See '**Contextual bandits.ipynb**' file in the Teams course page.

<u>How to sample weights (parameters of the LR)?</u>

```
def get_sampled_weights(self):
    w = np.random.normal(self.m, self.alpha * self.q**(-1/2))
    return w
```

✅ **Intuition**

- The model believes the best weight values are around m.
- But it is **not 100% certain**, so it samples a random perturbation.
- If the model is confident → **q is large** → std is small → samples stay close to m
- If the model is uncertain → **q is small** → std is large → exploration increases
- This randomness gives **exploration** in Thompson Sampling.

Implementing the Regularized LR

How to calculate loss?

```
def loss(self, w, *args):
    X, y = args
    n = len(y)
    regularizer = 0.5 * np.dot(self.q, (w - self.m)**2)
    pred_loss = sum([np.log(1 + np.exp(np.dot(w, X[j])))
                                - y[j] * np.dot(w, X[j]) for j in range(n)])
    return regularizer + pred_loss
```

Component	Purpose	Depends on
Prediction Loss (binary cross-entropy loss, or log-loss)	Measure how well the model matches the click data	y, X, w
Regularization	Prevent extreme weights and incorporate prior belief	w, m, q

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementing the Regularized LR

How to fit using loss?

```
def fit(self, X, y):
    if y:
        X = np.array(X)
        y = np.array(y)
        minimization = minimize(self.loss,
                                self.w,
                                args=(X, y),
                                method="L-BFGS-B",
                                bounds=[(-10,10)]*3 + [(-1, 1)],
                                options={'maxiter': 50})
        self.w = minimization.x
        self.m = self.w
        p = (1 + np.exp(-np.matmul(self.w, X.T)))**(-1)
        self.q = self.q + np.matmul(p * (1 - p), X**2)
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementing the Regularized LR

How to fit using loss?

Inputs:

- X → context features (device, location, age, etc.)
- y → click labels (1 = click, 0 = no click)

Steps:

1. Convert X and y to NumPy arrays.
2. Minimize the **loss function** (pred_loss + regularizer) using scipy.optimize.minimize.
3. Use bounds on weights w to avoid numerical overflow:
    - Binary/context features: [-10, +10]
    - Age: [-1, +1]
4. Update model weights: self.m = minimization.x

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementing the Regularized LR

How to fit using loss?

Batch updates: Model is updated after a batch of observations, not every single click.

Bayesian perspective:
Previous w values serve as the **prior** for the next batch.
Posterior mean updated → new self.m.

Uncertainty update:

```
p = 1 / (1 + exp(-X @ w)) # predicted probability of a click
self.q += np.matmul(p * (1 - p), X**2) # uncertain (0.5) p implies higher q
```

Updates diagonal covariance (precision) for Thompson sampling.

Implementing the Regularized LR

How to deal with upper confidence bounds? (just one of the explorations methods we will use)

```
def calc_sigmoid(self, w, context):
    return 1 / (1 + np.exp(-np.dot(w, context)))

def get_ucb(self, context):
    pred = self.calc_sigmoid(self.m, context)
    confidence = self.alpha * np.sqrt(np.sum(np.divide(np.array(context)**2, self.q)))
    ucb = pred + confidence
    return ucb
```

Code Variable	Actually Represents	Effect on UCB
pred	mean prediction (CTR estimate)	exploitation
confidence (higher values explore)	uncertainty width	exploration
ucb = pred + confidence	prediction + uncertainty bonus	optimistic action selection

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementing the Regularized LR

Implement two types of prediction methods, one using the mean values parameter estimates and the other using the sampled parameters to be used with Thompson sampling):

```
def get_prediction(self, context):
    return self.calc_sigmoid(self.m, context)

def sample_prediction(self, context):
    w = self.get_sampled_weights()
    return self.calc_sigmoid(w, context)
```

Objective – regret minimization

Regret measures **how much we lose** by not always picking the *optimal* action.

- Regret at step $k = p_{a^*}(x_k) - p_a(x_k)$

Symbol	Meaning
x_k	Context of user (k)
a^*	Best possible action (highest expected CTR)
a	Action our algorithm actually selected
p	Expected click-through rate (CTR)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Total (Cumulative) Regret

Regret measures **how much we lose** by not always picking the *optimal* action.

$$\text{Total regret} = \sum_{\mathrm{k}=1}^{\mathrm{K}} p_{a^*}(x_k) - p_a(x_k)$$

- Minimum regret at any step = 0
- Good algorithms → regret grows slowly
- We expect decelerating cumulative regret (curve bends and flattens)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

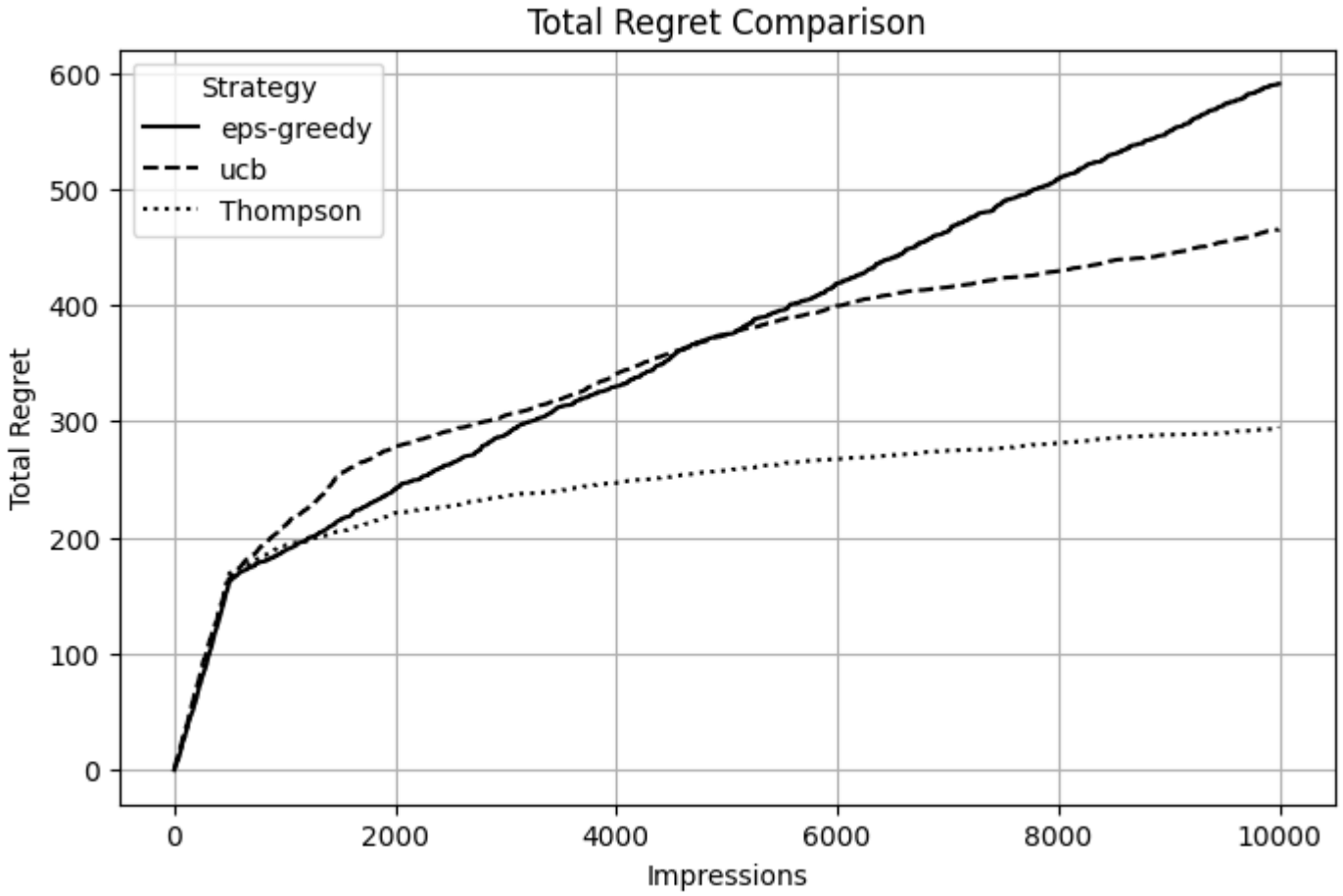
Code Used to Compute Regret

```python
def calculate_regret(ug, context, ad_options, ad):
    # 1. Compute true CTR values for all ads under this context
    action_values = {a: ug.logistic(ug.beta[a], context) for a in ad_options}

    # 2. Find the best possible ad (ground truth)
    best_action = max(action_values, key=action_values.get)

    # 3. Regret = best CTR - chosen CTR
    regret = action_values[best_action] - action_values[ad]

    return regret, best_action
```

Note that we are able to calculate the regret because we have access to the true action values (expected CTRs), which would not be the case in reality (although regret can still be estimated).

Solving the online advertising problem

See ‘**Contextual bandits.ipynb**’ file in the Teams course page.

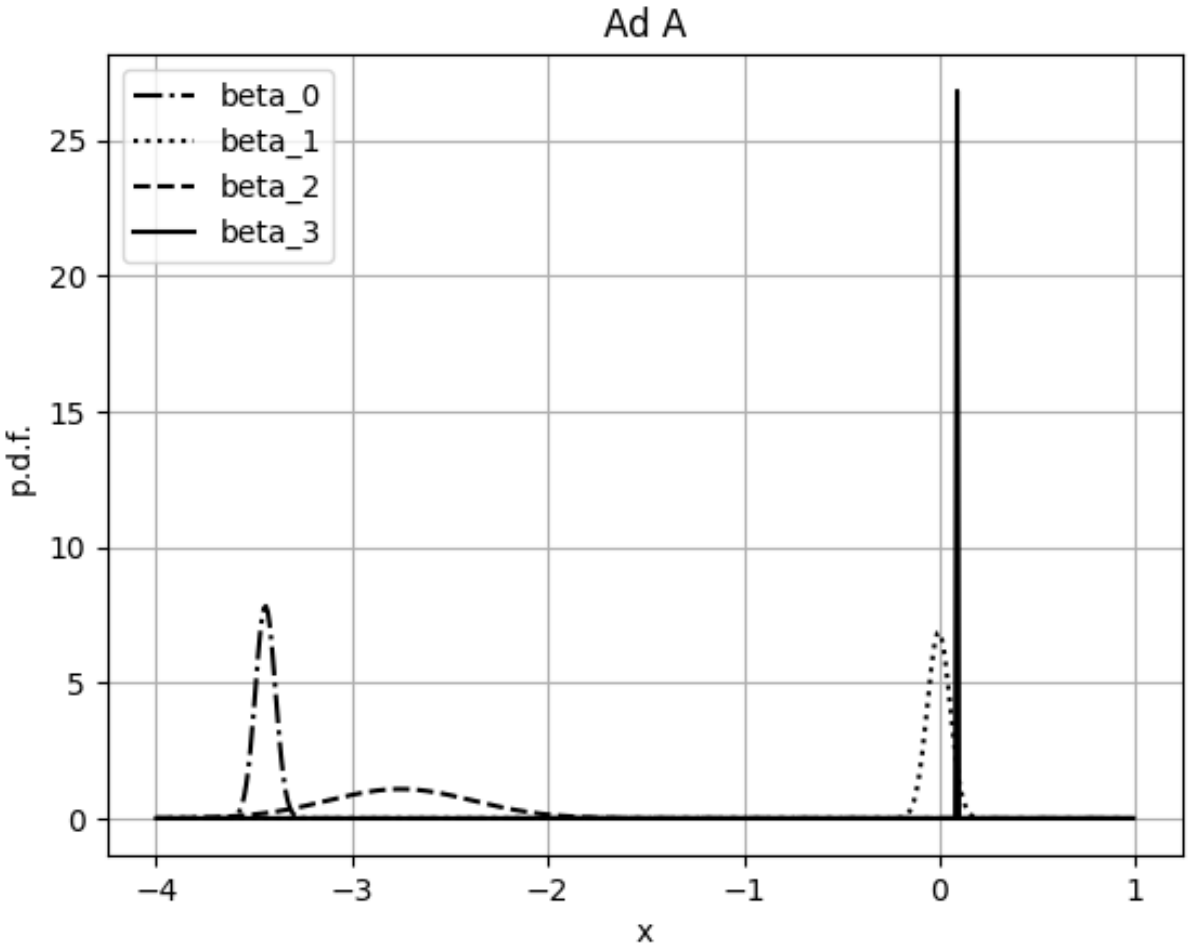


Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Solving the online advertising problem

- We clearly see that Thompson sampling is outperforming the ε-greedy and upper confidence bound (UCB) strategies as the total regret decelerates over time faster than the other two.
- Note that we have not tuned the exploration rate for ε-greedy and alpha for UCB, which could have led to better performances.
- But this is exactly the point: Thompson sampling provides a very effective exploration strategy pretty much out of the box.

Solving the online advertising problem - visualizing the parameter estimates



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Next Week

- Using function approximation for action
- Dynamic ad inventory
- Deep Neural Network (DNN) and dropout regularization based approximation